

Informe Entregable

Bases de Datos NoSQL

Base de datos NoSQL de enfermedades dermatológicas

Integrantes:

Blanca Lucy García García – 46143
Alejandra Montoya Rojas – 36751
Manuela Vallejo Toro – 28239

Universidad de Caldas

Docente:
Andrés Paolo Castaño

Octubre de 2026

ÍNDICE

1. Universo del discurso **1**
- 1.1. Base de datos NoSQL de enfermedades dermatológicas . . . 1
- 1.2. Delimitación del dominio y selección de afecciones dermatológicas . . . 1
  - 1.2.1. Enfermedades inflamatorias y autoinmunes . . . 2
  - 1.2.2. Enfermedades infecciosas . . . 2
  - 1.2.3. Trastornos de los anexos cutáneos . . . 2
  - 1.2.4. Enfermedades neoplásicas: tumores y cáncer de piel . . . 2
  - 1.2.5. Enfermedades genéticas o hereditarias . . . 3
  - 1.2.6. Alcance de la selección . . . 3

2. Análisis de requerimientos **3**
- 2.1. Requerimientos funcionales . . . 3
- 2.2. Requerimientos no funcionales . . . 4

3. Justificación del uso de NoSQL **5**

4. Comparación entre *embedding* y *referencing* **6**
- 4.1. Comparación de las estrategias . . . 6
- 4.2. Aplicación preliminar al dominio dermatológico . . . 7

5. Diseño del esquema de la base de datos **8**
- 5.1. Definición de colecciones con ejemplos de documentos . . . 8
  - 5.1.1. Colección `diagnosticos_dermatologicos` . . . 8
  - 5.1.2. Colección `sintomas` . . . 8
  - 5.1.3. Colección `medicamentos` . . . 9
  - 5.1.4. Colección `tratamientos` . . . 9
  - 5.1.5. Colección `diagnosticos_asignados` . . . 10
  - 5.1.6. Colección `pacientes` . . . 12
  - 5.1.7. Colección `historias_medicas` . . . 13
  - 5.1.8. Colección `imagenes` . . . 14
  - 5.1.9. Colección `resultados_laboratorio` . . . 15
  - 5.1.10. Colección `documentos_conocimiento` . . . 15
  - 5.1.11. Colección `chunks` . . . 16
- 5.2. Estrategias de indexación planificadas . . . 17
  - 5.2.1. Consideraciones de implementación . . . 19
- 5.3. Reglas de validación del esquema . . . 19

6. Configuración de Entorno **20**
- 6.1. Cluster MongoDB configurado *Atlas* . . . 20
- 6.2. Scripts de inicialización . . . 21
- 6.3. Conexión verificada desde aplicación . . . 22
- 6.4. Script de Colecciones Ejemplo insertadas en Mongo . . . 22
- 6.5. Implementar la indexación de las colecciones. . . . 28

7. Plan de experimento de *chunking* **32**
- 7.1. Definición de las estrategias a comparar y sus parámetros . . . 32
- 7.2. Conjunto de 10 consultas de prueba seleccionadas . . . 33

1. Universo del discurso

1.1. Base de datos NoSQL de enfermedades dermatológicas

El proyecto consiste en el diseño de una base de datos no relacional orientada al almacenamiento, consulta y análisis de información relacionada con enfermedades, afecciones y lesiones dermatológicas. Su propósito es centralizar información sobre sus características, síntomas, manifestaciones visuales, tratamientos, medicamentos asociados y contraindicaciones, así como información clínica relevante de los pacientes.

La base de datos distinguirá entre los **diagnósticos dermatológicos establecidos**, entendidos como entidades que describen las características, manifestaciones y comportamiento de una enfermedad o afección, y los **diagnósticos asignados a pacientes**, que corresponden a registros particulares dentro de su historia médica. Esta distinción permitirá estudiar las afecciones independientemente de los casos individuales y, al mismo tiempo, relacionarlas con los antecedentes clínicos de los pacientes cuando corresponda.

El sistema permitirá consultar posibles afecciones a partir de uno o varios síntomas, identificar tratamientos asociados a un diagnóstico, explorar similitudes entre imágenes de lesiones cutáneas y analizar posibles relaciones secuenciales entre diagnósticos registrados en las historias médicas. Asimismo, permitirá identificar posibles incompatibilidades entre medicamentos, considerando las alergias de los pacientes, sus tratamientos actuales y las contraindicaciones conocidas.

La información almacenada incluirá **datos estructurados**, como síntomas, diagnósticos, medicamentos, tratamientos, alergias, historias médicas y resultados de laboratorio; y **datos no estructurados**, como imágenes de lesiones cutáneas, descripciones de colores y texturas, e informes de exámenes en formatos JSON o TXT. Estos últimos podrán procesarse y vectorizarse para facilitar la búsqueda semántica y multimodal, recuperar información relevante y generar respuestas fundamentadas mediante un sistema de recuperación aumentada por generación (RAG).

La solución utilizará **MongoDB** como sistema gestor de base de datos, aprovechando su modelo documental flexible y sus capacidades de búsqueda e indexación para relacionar la información clínica, recuperar conocimiento y responder a las consultas definidas para el dominio dermatológico.

1.2. Delimitación del dominio y selección de afecciones dermatológicas

Debido a la amplitud del campo de la salud, el proyecto se centrará en un conjunto delimitado de enfermedades, trastornos y afecciones dermatológicas. Esta selección permitirá construir una base de conocimiento especializada, con información suficiente para relacionar síntomas, características clínicas, manifestaciones visuales, diagnósticos, tratamientos y medicamentos.

Las afecciones contempladas se organizarán en cinco categorías principales, de acuerdo con su naturaleza y características generales.

1.2.1. Enfermedades inflamatorias y autoinmunes

Comprenden afecciones relacionadas con procesos inflamatorios de la piel o alteraciones del sistema inmunitario que pueden afectar los tejidos cutáneos.

- **Dermatitis y eccemas:** afecciones que pueden provocar inflamación, enrojecimiento, descamación y picazón. Se considerarán la dermatitis atópica y la dermatitis por contacto.
- **Psoriasis:** enfermedad inflamatoria crónica que puede producir placas de piel engrosada cubiertas de escamas.
- **Lupus eritematoso cutáneo:** afección autoinmunitaria que puede causar lesiones e inflamación en la piel, incluyendo erupciones en el rostro.
- **Vitíligo:** trastorno caracterizado por la pérdida de pigmentación en determinadas zonas de la piel, que produce manchas más claras.

1.2.2. Enfermedades infecciosas

Incluyen afecciones cutáneas causadas por microorganismos o parásitos. Se clasificarán según el agente causante.

- **Bacterianas:** como el impétigo, la celulitis infecciosa y la foliculitis.
- **Virales:** como el herpes simple, el herpes zóster y las verrugas causadas por el virus del papiloma humano.
- **Micóticas o fúngicas:** como la tiña, incluido el pie de atleta, y la candidiasis cutánea.
- **Parasitarias:** como la escabiosis o sarna.

1.2.3. Trastornos de los anexos cutáneos

Comprenden alteraciones que afectan estructuras asociadas a la piel, como los folículos pilosos y las glándulas sebáceas.

- **Acné:** trastorno relacionado con la obstrucción de los folículos y procesos inflamatorios que pueden generar comedones, pápulas y pústulas.
- **Rosácea:** afección que puede producir enrojecimiento persistente del rostro, vasos sanguíneos visibles y lesiones inflamatorias.
- **Alopecia:** pérdida parcial o total del cabello, que puede tener diferentes causas, incluyendo factores genéticos, inmunitarios o inflamatorios.

1.2.4. Enfermedades neoplásicas: tumores y cáncer de piel

Comprenden crecimientos anormales de células en la piel. Para el proyecto se distinguirán las lesiones y los tumores benignos de las neoplasias malignas.

- **Benignas:** se considerarán lesiones como los nevos o lunares y las queratosis benignas. Los lipomas podrán incluirse cuando se contemplen tumores benignos de tejidos subcutáneos relacionados con el dominio dermatológico.

- **Malignas:** se incluirán el melanoma y los carcinomas cutáneos no melanoma, principalmente el carcinoma basocelular y el carcinoma espinocelular.

1.2.5. Enfermedades genéticas o hereditarias

Incluyen trastornos asociados a alteraciones genéticas que afectan la estructura, la pigmentación o la resistencia de la piel.

- **Ictiosis:** grupo de trastornos que pueden provocar sequedad intensa, engrosamiento y descamación de la piel.
- **Epidermólisis ampollosa:** grupo de enfermedades caracterizadas por una fragilidad cutánea que facilita la aparición de ampollas y heridas ante traumatismos leves.

1.2.6. Alcance de la selección

Las categorías y afecciones anteriores constituyen el alcance inicial del dominio dermatológico de la base de datos. La información recopilada se utilizará para establecer relaciones entre síntomas, enfermedades, manifestaciones visuales, tratamientos y medicamentos, así como para explorar similitudes entre imágenes y posibles secuencias de diagnósticos en las historias médicas.

La selección no pretende abarcar la totalidad de las enfermedades dermatológicas ni sustituir la evaluación de un profesional de la salud. Su finalidad es establecer un dominio de conocimiento acotado y suficientemente diverso para desarrollar y evaluar las funcionalidades de consulta, recuperación semántica y generación de respuestas del sistema NoSQL con MongoDB.

2. Análisis de requerimientos

El sistema deberá permitir almacenar, consultar y relacionar información sobre enfermedades y afecciones dermatológicas, sus síntomas, manifestaciones visuales, tratamientos, medicamentos y contraindicaciones, así como información clínica asociada a los pacientes. Para ello, se establecen los siguientes requerimientos funcionales y no funcionales.

2.1. Requerimientos funcionales

RF-01. Consulta de afecciones por síntomas. El sistema deberá permitir consultar posibles afecciones dermatológicas a partir de uno o varios síntomas, recuperando aquellas que presenten características compatibles con los criterios de búsqueda.

RF-02. Consulta de tratamientos. El sistema deberá permitir consultar los tratamientos asociados a una afección dermatológica y recuperar información sobre los medicamentos relacionados, sus indicaciones y precauciones documentadas.

RF-03. Búsqueda basada en imágenes. El sistema deberá permitir recuperar imágenes visualmente similares y explorar posibles afecciones relacionadas con una imagen de entrada, mediante técnicas de procesamiento de imágenes y búsqueda vectorial.

RF-04. Análisis de secuencias de diagnósticos. El sistema deberá permitir analizar las historias médicas para identificar patrones de sucesión entre diagnósticos registrados en diferentes momentos, con el propósito de explorar posibles relaciones temporales o estadísticas.

RF-05. Identificación de incompatibilidades medicamentosas. El sistema deberá permitir contrastar las alergias conocidas de un paciente, los medicamentos que utiliza y sus tratamientos actuales con las contraindicaciones e interacciones documentadas, con el fin de identificar posibles riesgos.

RF-06. Almacenamiento de información heterogénea. El sistema deberá permitir almacenar información estructurada, como síntomas, afecciones, medicamentos y resultados de laboratorio, así como información no estructurada, incluyendo imágenes de lesiones cutáneas, descripciones de colores y texturas e informes de laboratorio en formatos JSON o TXT.

RF-07. Recuperación semántica de información. El sistema deberá permitir recuperar información relevante a partir de consultas expresadas en lenguaje natural, utilizando representaciones vectoriales del contenido y mecanismos de búsqueda semántica.

RF-08. Generación de respuestas fundamentadas. El sistema deberá integrar un mecanismo de recuperación aumentada por generación (RAG), que utilice información recuperada de la base de conocimiento como contexto para generar respuestas relacionadas con el dominio dermatológico.

RF-09. Consulta de información clínica histórica. El sistema deberá permitir consultar los antecedentes clínicos de los pacientes, incluyendo las historias médicas, los exámenes realizados, los diagnósticos asignados y los tratamientos registrados.

2.2. Requerimientos no funcionales

RNF-01. Flexibilidad del modelo de datos. La estructura de almacenamiento deberá permitir representar diferentes tipos de afecciones y sus características particulares, sin exigir que todos los documentos contengan exactamente los mismos atributos.

RNF-02. Eficiencia de consulta. El sistema deberá contar con mecanismos de indexación y estrategias de recuperación adecuadas para facilitar las consultas por atributos, las búsquedas textuales y las búsquedas vectoriales.

RNF-03. Integridad y consistencia de la información. La aplicación deberá mantener relaciones coherentes entre los datos almacenados, validar las referencias entre documentos y evitar inconsistencias en la información clínica.

RNF-04. Trazabilidad de la información. Las respuestas generadas deberán poder relacionarse con las fuentes documentales recuperadas, de manera que sea posible verificar su fundamento.

RNF-05. Alcance clínico responsable. El sistema deberá presentar los resultados como información de apoyo y posibilidades que requieren interpretación profesional, sin sustituir el diagnóstico ni las decisiones de tratamiento realizadas por personal de salud.

3. Justificación del uso de NoSQL

El proyecto requiere una solución de almacenamiento capaz de gestionar información dermatológica de distinta naturaleza y complejidad. Los datos contemplados incluyen entidades estructuradas, como síntomas, afecciones, medicamentos y tratamientos, así como imágenes, descripciones visuales e informes de laboratorio que pueden presentar formatos y estructuras variables.

En este contexto, se propone utilizar MongoDB como sistema gestor de base de datos no relacional, debido a su modelo documental y a su capacidad para almacenar documentos con estructuras flexibles. Esta característica resulta pertinente para representar afecciones dermatológicas con atributos diferentes, evitando imponer una estructura completamente uniforme a entidades que pueden requerir información específica según su naturaleza.

El modelo documental también permite combinar información incrustada y referencias entre documentos, de acuerdo con las necesidades de consulta, reutilización y actualización de los datos. De esta manera, es posible mantener juntas las características que suelen consultarse en conjunto y separar la información que necesita administrarse de forma independiente.

Adicionalmente, el proyecto requiere recuperar información mediante consultas tradicionales, búsqueda textual y similitud semántica. El uso de MongoDB Atlas permite incorporar capacidades de búsqueda vectorial que, junto con modelos de *embeddings* y técnicas de procesamiento de lenguaje natural e imágenes, pueden utilizarse para desarrollar funcionalidades de recuperación de información multimodal.

La integración de estas capacidades con un sistema RAG permitirá recuperar fragmentos relevantes de la base de conocimiento y utilizarlos como contexto para generar respuestas relacionadas con síntomas, afecciones, tratamientos y otras consultas del dominio dermatológico.

Por último, Aunque una base de datos relacional podría utilizarse para gestionar la información dermatológica del proyecto, el modelo documental de MongoDB se ajusta mejor a la heterogeneidad de los datos y a la necesidad de representar afecciones con atributos variables. En un modelo relacional, esta información normalmente requeriría definir tablas y relaciones previamente, lo que podría implicar ajustes en el esquema cuando se incorporen nuevas características específicas de determinadas afecciones. En MongoDB, estas variaciones pueden representarse mediante documentos con estructuras diferentes dentro de una misma colección, manteniendo los atributos comunes y permitiendo incorporar campos específicos cuando sea necesario.

Asimismo, el modelo documental facilita agrupar en un mismo documento los datos que suelen consultarse conjuntamente y utilizar referencias para la información compartida o administrada de manera independiente. Esto permite adaptar la organización de los datos a los patrones de consulta del sistema. Si bien las bases de datos relacionales también ofrecen mecanismos para gestionar datos semiestructurados y pueden integrarse con soluciones de búsqueda vectorial, MongoDB proporciona un modelo documental y capacidades de búsqueda integradas que resultan convenientes para los requerimientos de recuperación de información del proyecto.

Por tanto, la elección de MongoDB no se fundamenta en que las bases de datos relacionales sean inadecuadas, sino en que su modelo documental ofrece una correspondencia más

directa con la variabilidad de los datos dermatológicos, la evolución del esquema y las necesidades de recuperación de información del sistema.

Por estas razones, MongoDB constituye una alternativa adecuada para el proyecto, al permitir combinar el almacenamiento documental con mecanismos de búsqueda e integración necesarios para desarrollar el sistema RAG.

4. Comparación entre *embedding* y *referencing*

En el modelado documental de MongoDB existen dos estrategias principales para representar información relacionada: *embedding* y *referencing*. La elección entre ambas depende de la naturaleza de los datos, la frecuencia con la que se consultan conjuntamente, su reutilización y su crecimiento esperado.

El ***embedding*** consiste en incorporar un documento o conjunto de atributos relacionados dentro de otro documento. Esta estrategia permite recuperar información relacionada en una misma consulta y resulta conveniente cuando los datos son dependientes, relativamente pequeños y suelen utilizarse en conjunto.

El ***referencing*** consiste en almacenar la información en documentos independientes y relacionarlos mediante identificadores. Esta estrategia favorece la reutilización de entidades compartidas, evita la duplicación innecesaria y facilita la administración de información que puede crecer o actualizarse de manera independiente.

4.1. Comparación de las estrategias

Cuadro 1: Comparación entre *embedding* y *referencing*

Criterio	***Embedding***	***Referencing***
Organización	Los datos relacionados se almacenan dentro de un mismo documento.	Los datos se almacenan en documentos independientes vinculados mediante identificadores.
Recuperación	Facilita obtener conjuntamente los datos incrustados en una consulta.	Puede requerir consultas adicionales o utilizar `$lookup` para combinar documentos.
Reutilización	Puede producir duplicación cuando la misma información se incorpora en varios documentos.	Facilita que múltiples documentos hagan referencia a una entidad compartida.
Actualización	Es sencillo actualizar datos contenidos en un mismo documento; la duplicación puede complicar su mantenimiento.	Permite actualizar una entidad centralizada sin modificar todos los documentos que la referencian.
Crecimiento	Es apropiado para información limitada; los documentos pueden volverse demasiado grandes si acumulan datos.	Facilita administrar entidades que crecen de manera independiente.

Continúa en la siguiente página

Cuadro 1: Comparación entre *embedding* y *referencing* (continuación)

Criterio	***Embedding***	***Referencing***
Relaciones entre entidades	Es conveniente para datos dependientes o que se consultan conjuntamente.	Resulta conveniente para entidades independientes y relaciones de reutilización, incluidas las de muchos a muchos.
Complejidad de consulta	Puede reducir la necesidad de combinar documentos.	Puede aumentar la complejidad de algunas consultas al requerir resolver referencias.
Consistencia	La información duplicada puede generar inconsistencias si no se actualiza correctamente.	Centralizar una entidad reduce la duplicación, aunque la aplicación debe validar que las referencias sean correctas.

Ninguna estrategia es universalmente superior. En nuestra base de datos vamos a combinar ambas, seleccionando la alternativa más apropiada para cada relación.

4.2. Aplicación preliminar al dominio dermatológico

A partir de las características y necesidades identificadas en el análisis, se establecen los siguientes criterios preliminares para orientar el futuro diseño del esquema.

Cuadro 2: Criterios preliminares de aplicación al dominio dermatológico

Casos en los que consideraríamos *embedding*	**Casos en los que consideraríamos *referencing***
Atributos complementarios que suelen consultarse junto con su entidad principal.	Síntomas que pueden estar asociados a múltiples afecciones dermatológicas.
Información de alergias, incluyendo la sustancia causante y la reacción registrada, vinculada al paciente.	Afecciones dermatológicas establecidas que deben mantenerse independientes de los pacientes a quienes se les asignan diagnósticos.
Características descriptivas de una imagen, como colores, texturas y región corporal.	Medicamentos que pueden formar parte de distintos tratamientos.
Contraindicaciones asociadas a un medicamento, cuando sean acotadas y se consulten habitualmente con su información.	Historias médicas que acumulan episodios clínicos a lo largo del tiempo.
Metadatos pequeños y dependientes que no necesitan administrarse como entidades independientes.	Imágenes e informes de laboratorio que deben conservarse y gestionarse como recursos independientes.
Información complementaria de un tratamiento que no tenga una reutilización independiente significativa.	Documentos o fragmentos de conocimiento que deban vincularse con sus fuentes originales para la recuperación RAG.

Como criterio general, se priorizará el *embedding* para información pequeña y dependiente

que se consulta conjuntamente, mientras que se considerará el *referencing* para entidades compartidas, independientes o con crecimiento propio. La combinación de ambas estrategias permitirá equilibrar eficiencia de consulta, reutilización de información y facilidad de mantenimiento.

5. Diseño del esquema de la base de datos

5.1. Definición de colecciones con ejemplos de documentos

5.1.1. Colección `diagnosticos_dermatologicos`

Propósito: almacenar los diagnósticos dermatológicos establecidos como entidades de conocimiento que describen una afección, sus características, síntomas y manifestaciones visuales. No representa un diagnóstico particular asignado a un paciente.

Embedding: se aplica a las características descriptivas que suelen consultarse junto con el diagnóstico, como la descripción y las manifestaciones visuales.

Referencing: se aplica a los síntomas, porque un mismo síntoma puede estar relacionado con varios diagnósticos, y a los tratamientos asociados, que se administran de manera independiente.

Ejemplo de documento:

```
{
  "_id": "ObjectId(...)",
  "nombre": "Psoriasis",
  "categoria": "Inflamatoria y autoinmune",
  "descripcion": "Afección inflamatoria crónica de la piel.",
  "sintomas_ids": [
    "ObjectId(sintoma_1)",
    "ObjectId(sintoma_2)"
  ],
  "manifestaciones_visuales": [
    "Placas rojizas",
    "Escamas blanquecinas"
  ],
  "tratamientos_ids": [
    "ObjectId(tratamiento_1)"
  ]
}
```

5.1.2. Colección `sintomas`

Propósito: almacenar los síntomas que pueden aparecer asociados a diferentes diagnósticos dermatológicos. Se mantiene esta colección independiente para evitar duplicar el mismo síntoma en varios diagnósticos.

Embedding: no se requiere *embedding* de otros documentos dentro de esta colección para el modelo básico propuesto; cada síntoma es un documento independiente.

Referencing: los documentos de `diagnosticos_dermatologicos` hacen referencia a los síntomas mediante `sintomas_ids`.

Ejemplo de documento:

```
{
  "_id": "ObjectId(...)",
  "nombre": "Picazón",
  "descripcion": "Sensación de irritación que provoca
    la necesidad de rascarse."
}
```

5.1.3. Colección `medicamentos`

Propósito: almacenar la información propia de cada medicamento, incluidas las indicaciones, contraindicaciones e interacciones documentadas. La colección permite que un medicamento se reutilice en diferentes tratamientos.

Embedding: se aplica a las contraindicaciones e interacciones son datos acotados que suelen consultarse junto con el medicamento.

Referencing: se aplica a otros medicamentos implicados en una interacción, si estos deben identificarse individualmente. Los tratamientos hacen referencia a los medicamentos mediante sus identificadores.

Ejemplo de documento:

```
{
  "_id": "ObjectId(...)",
  "nombre": "Medicamento de ejemplo",
  "principio_activo": "Principio activo de ejemplo",
  "indicaciones": [
    "Indicación documentada"
  ],
  "contraindicaciones": [
    "Contraindicación documentada"
  ],
  "interacciones": [
    {
      "medicamento_relacionado_id": "ObjectId(...)",
      "descripcion": "Interacción documentada"
    }
  ]
}
```

5.1.4. Colección `tratamientos`

Propósito: almacenar los tratamientos establecidos para los diagnósticos dermatológicos, como entidades de conocimiento que describen sus indicaciones, precauciones y medicamentos. Al igual que `diagnosticos_dermatologicos`, esta colección es independiente de los pacientes: no representa un tratamiento recibido por una persona, sino el tratamiento de referencia asociado a uno o más diagnósticos.

Embedding: se aplica a la información complementaria propia del tratamiento, como indicaciones y precauciones, y a las pautas generales de cada medicamento. Estas pautas se guardan dentro del elemento correspondiente de la lista `medicamentos`, en el arreglo `pautas_generales`, con una entrada por grupo poblacional (por ejemplo, adulto o pediátrico), porque la cantidad y la frecuencia de un mismo medicamento pueden ser diferentes según el grupo. Son valores generales de referencia, no ajustados a un paciente particular.

Referencing: se aplica a los diagnósticos dermatológicos a los que pertenece el tratamiento (`diagnosticos_ids`) y a los medicamentos (`medicamento_id`), dado que un mismo medicamento puede formar parte de varios tratamientos. Las variaciones propias de cada paciente no se guardan aquí, sino en la colección `tratamientos_asignados`.

Ejemplo de documento:

```
{
  "_id": "ObjectId(...)",
  "nombre": "Tratamiento tópico de ejemplo",
  "descripcion": "Tratamiento de aplicación local.",
  "diagnosticos_ids": [
    "ObjectId(diagnostico_1)"
  ],
  "indicaciones": [
    "Indicación documentada"
  ],
  "precauciones": [
    "Precaución documentada"
  ],
  "medicamentos": [
    {
      "medicamento_id": "ObjectId(medicamento_1)",
      "pautas_generales": [
        {
          "grupo_poblacional": "Adulto",
          "cantidad": "Cantidad general indicada en la pauta",
          "frecuencia": "Frecuencia general indicada en la pauta"
        },
        {
          "grupo_poblacional": "Pediátrico",
          "cantidad": "Cantidad general indicada en la pauta",
          "frecuencia": "Frecuencia general indicada en la pauta"
        }
      ]
    }
  ]
}
```

5.1.5. Colección `diagnosticos_asignados`

Propósito: almacenar los diagnósticos que se asignan a un paciente en un momento determinado, junto con los tratamientos que se le asignan a raíz de cada uno. Esta colección conserva la distinción entre el diagnóstico establecido y el diagnóstico asigna-

do: `diagnosticos_dermatologicos` describe la afección como entidad de conocimiento, mientras que `diagnosticos_asignados` documenta su asignación a un paciente. Del mismo modo, `tratamientos` describe el tratamiento establecido y el elemento embebido `tratamientos_asignados` documenta su aplicación en ese caso.

Embedding: se aplica al `tratamientos_asignados`, porque cada tratamiento asignado depende del diagnóstico asignado al que responde y suele consultarse junto con él. Cada elemento contiene la lista de medicamentos con la cantidad y la frecuencia ajustadas al paciente, las fechas de inicio y fin, el indicador `vigente` y las observaciones, que son datos pequeños propios de esa asignación.

Referencing: se aplica a `diagnostico_id`, que identifica el diagnóstico establecido de referencia; a `paciente_id`; a `historia_medica_id` y al episodio clínico en el que se asignó; a `tratamiento_id`, que identifica el tratamiento establecido de referencia en cada tratamiento asignado; y a `medicamento_id` dentro de cada medicamento.

Ejemplo de documento:

```
{
  "_id": "ObjectId(...)",
  "paciente_id": "ObjectId(paciente_1)",
  "historia_medica_id": "ObjectId(historia_1)",
  "episodio_fecha": "ISODate(...)",
  "diagnostico_id": "ObjectId(diagnostico_1)",
  "fecha_diagnostico": "ISODate(...)",
  "vigente": true,
  "observaciones": "Observaciones propias del caso",
  "tratamientos_asignados": [
    {
      "tratamiento_id": "ObjectId(tratamiento_1)",
      "fecha_inicio": "ISODate(...)",
      "fecha_fin": null,
      "vigente": true,
      "medicamentos": [
        {
          "medicamento_id": "ObjectId(medicamento_1)",
          "cantidad": "Cantidad indicada para este paciente",
          "frecuencia": "Frecuencia indicada para este paciente"
        }
      ],
      "observaciones": "Ajustes o indicaciones propias del caso"
    }
  ]
}
```

Interpretación de los campos:

- `vigente` (del diagnóstico): `true` significa que el diagnóstico se considera vigente en el registro clínico del paciente; `false` significa que ya no se considera vigente.
- `tratamientos_asignados`: arreglo de tratamientos aplicados al paciente a partir de ese diagnóstico. Puede estar vacío si aún no se ha asignado ningún tratamiento.

- `cantidad` y `frecuencia`: corresponden a la pauta aplicada al paciente, que puede partir de la pauta general del grupo poblacional que le corresponde en `tratamientos` y modificarse según el caso.
- `fecha_fin`: será `null` mientras el tratamiento siga en curso, o será una fecha pautada por el personal de salud en caso de ser un tratamiento temporal o de duración corta.
- `vigente` (del tratamiento): indica si ese tratamiento continúa vigente para el paciente dentro del diagnóstico asignado. Ya que podría seguir vigente el diagnostico pero no el tratamiento en cuestión. Un paciente puede tener varios `tratamientos_asignados` para un mismo `diagnosticos_asignados` con vigencias diferentes.

5.1.6. Colección `pacientes`

Propósito: almacenar la información de identificación y residencia de cada paciente, sus alergias y las referencias a sus diagnósticos asignados e historias clínicas. El estado de vigencia de los diagnósticos y tratamientos ya no se guarda aquí, sino en la colección `diagnosticos_asignados`.

Embedding: se aplica a las alergias, porque la sustancia causante y la reacción registrada son datos pequeños que dependen del paciente.

Referencing: se aplica a los diagnósticos asignados (`diagnosticos_asignados_ids`) y a las historias clínicas (`historias_clinicas_ids`). Las historias clínicas y los diagnósticos asignados se administran en colecciones independientes para soportar su crecimiento. Los diagnósticos asignados llevan embebidos sus tratamientos, por lo que no es necesario referenciar los tratamientos desde el paciente. Pese a eso (`diagnosticos_asignados_ids`) ya se encuentra embebido en (`historias_clinicas_ids`), pero para facilidad de consultas se decide referencar ambos de forma independiente en esta colección.

Ejemplo de documento:

```
{
  "_id": "ObjectId(...)",
  "codigo_paciente": "PAC-001",
  "nombre_completo": "Paciente de ejemplo",
  "fecha_nacimiento": "ISODate(...)",
  "lugar_residencia": {
    "pais": "Colombia",
    "ciudad": "Manizales"
  },
  "fecha_defuncion": null,
  "alergias": [
    {
      "sustancia": "Sustancia de ejemplo",
      "reaccion": "Reacción registrada"
    }
  ],
  "diagnosticos_asignados_ids": [
    "ObjectId(diagnostico_asignado_1)",
    "ObjectId(diagnostico_asignado_2)"
  ],
```

```
  "historias_clinicas_ids": [
    "ObjectId(historia_1)"
  ]
}
```

Interpretación de los campos:

- `fecha_defuncion`: será `null` si el paciente sigue vivo y tendrá una fecha si se registra su fallecimiento.
- `diagnosticos_asignados_ids`: contiene referencias a los documentos del diagnostico propio del paciente, tanto vigentes como no vigentes. El estado de cada uno se consulta en su propio documento.
- `historias_clinicas_ids`: contiene referencias a los documentos de historias clínicas del paciente.

5.1.7. Colección `historias_medicas`

Propósito: almacenar la historia clínica de un paciente y los episodios registrados a lo largo del tiempo, incluidos diagnósticos asignados, observaciones y tratamientos asociados.

Embedding: se aplica a los episodios y a sus observaciones clínicas, siempre que el volumen de la historia se mantenga dentro de límites razonables, se sobre-entenderá que habrán historias que creceran mucho más que otras, dependiendo de la situación de cada paciente.

Referencing: se aplica al paciente propietario de la historia y a los diagnosticos asignados establecidos y tratamientos relacionados con cada episodio. Las imágenes y los resultados de laboratorio se mantienen en colecciones independientes; los episodios pueden guardar sus identificadores.

Ejemplo de documento:

```
{
  "_id": "ObjectId(...)",
  "paciente_id": "ObjectId(paciente_1)",
  "episodios": [
    {
      "fecha": "ISODate(...)",
      "diagnosticos_asignados_ids": [
        "ObjectId(diagnostico_1)"
        ],
      "observaciones": "Manifestaciones observadas",
      "imagenes_ids": [
        "ObjectId(imagen_1)"
      ],
      "resultados_laboratorio_ids": [
        "ObjectId(resultado_1)"
      ]
    }
  ]
```

```
}
```

La distinción entre el diagnóstico establecido y el diagnóstico asignado se conserva: `diagnostico_id` identifica la entidad dermatológica de referencia, mientras que la presencia de ese identificador en un episodio documenta que se asignó al paciente en un momento determinado.

Si el historial llegara a crecer mucho o requiriera administrar episodios individualmente, sería conveniente evaluar una colección independiente para los episodios. Para el diseño inicial, el *embedding* responde al criterio de mantener juntos los datos que se consultan como parte de la historia clínica.

5.1.8. Colección `imagenes`

Propósito: almacenar metadatos y representaciones vectoriales de imágenes de lesiones cutáneas para permitir la búsqueda de imágenes visualmente similares y explorar posibles diagnósticos relacionados.

Embedding: se aplica a las características descriptivas de la imagen, como colores, texturas y región corporal. También se guarda el vector de características visuales, embedding visual, generado por el modelo de procesamiento de imágenes.

Referencing: se aplica a `diagnostico_id` y, cuando la imagen pertenece a la historia clínica de un paciente registrado, a `paciente_id` y al episodio clínico correspondiente.

El campo `paciente_id` puede ser `null`. Si la imagen es un ejemplo general asociado a un diagnóstico y no corresponde a un paciente de la base de datos. En cambio, si `paciente_id` tiene un valor, la imagen debe estar relacionada con un paciente registrado y con el episodio clínico correspondiente.

Ejemplo de documento:

```
{
  "_id": "ObjectId(...)",
  "paciente_id": null,
  "historia_medica_id": null,
  "episodio_fecha": null,
  "diagnostico_id": "ObjectId(diagnostico_1)",
  "archivo": "imagen_referencia_001.jpg",
  "descripcion_visual": {
    "colores": ["Rojizo", "Blanquecino"],
    "texturas": ["Descamación"],
    "region_corporal": "Codo"
  },
  "embedding_visual": [0.12, -0.08, 0.31],
  "fecha_registro": "ISODate(...)"
}
```

PREGUNTAR A PAOLO: La propuesta inicial es generar un *embedding* visual por imagen y almacenarlo en `embedding_visual`, sin dividir la imagen en *chunks* de texto. Si posteriormente se decide procesar regiones o recortes de una imagen, podrían generarse

vectores adicionales para esas regiones, pero esa sería una ampliación del diseño y no un requisito establecido

5.1.9. Colección `resultados_laboratorio`

Propósito: almacenar los resultados de exámenes e informes de laboratorio asociados a pacientes. Esta colección permite conservar resultados estructurados o semiestructurados, incluidos datos procedentes de informes JSON o TXT, sin incorporar todo el contenido dentro de la historia médica.

Embedding: se aplica a los datos que componen un resultado cuando son pequeños y dependen de ese examen, como el parámetro evaluado, su valor y las observaciones.

Referencing: se aplica al paciente y, cuando corresponda, a la historia clínica y al episodio al que pertenece el examen.

Ejemplo de documento:

```
{
  "_id": "ObjectId(...)",
  "paciente_id": "ObjectId(paciente_1)",
  "historia_medica_id": "ObjectId(historia_1)",
  "fecha": "ISODate(...)",
  "tipo_examen": "Examen de laboratorio",
  "formato": "JSON",
  "resultado": {
    "parametro": "Parámetro de ejemplo",
    "valor": "Resultado registrado",
    "observaciones": "Observación del informe"
  }
}
```

La estructura de `resultado` puede adaptarse al tipo de examen, conservando la flexibilidad documental de MongoDB.

5.1.10. Colección `documentos_conocimiento`

Propósito: almacenar los documentos textuales que constituyen el contenido de conocimiento utilizado por el sistema RAG. Pueden incluir información sobre diagnósticos dermatológicos, síntomas, manifestaciones, tratamientos y medicamentos. Su función principal es conservar el texto que será procesado para la recuperación de información y la generación de respuestas fundamentadas.

Esta colección no representa pacientes ni episodios clínicos. Tampoco constituye una colección separada de fuentes: almacena directamente los documentos de conocimiento que se procesan.

Embedding: se aplica a los metadatos pequeños propios del documento, como su nombre, tipo e idioma. El contenido textual completo se conserva en `contenido_texto`; sus representaciones vectoriales se guardan en la colección `chunks`.

Referencing: la colección `chunks` referencia cada documento mediante `doc_id`. Así, cuando se recupera un fragmento, el sistema puede identificar el documento del que

procede y consultar su contenido asociado.

Ejemplo de documento:

```
{
  "_id": "ObjectId(...)",
  "nombre_archivo": "documento_dermatologico.txt",
  "tipo_documento": "Texto",
  "idioma": "es",
  "contenido_texto": "Contenido sobre características,
    síntomas, manifestaciones o tratamientos
    dermatológicos.",
  "fecha": "ISODate(...)"
}
```

El flujo de funcionamiento es el siguiente: se almacena el documento, se extrae o prepara su contenido textual, se divide en fragmentos, se generan *embeddings* para dichos fragmentos y se guardan en `chunks`. Ante una consulta, el sistema recupera los fragmentos pertinentes y utiliza su contenido como contexto para generar una respuesta mediante RAG.

5.1.11. Colección `chunks`

Propósito: almacenar los fragmentos de texto derivados de `documentos_conocimiento`, junto con su estrategia de división, el *embedding* textual y los datos necesarios para identificar su posición y documento original. Esta colección permite comparar estrategias de *chunking* y realizar recuperación semántica para el sistema RAG.

Embedding: se aplica a cada fragmento de texto mediante el modelo de *embeddings* seleccionado, `jina-clip-v2`. Se guarda el vector en `embedding`, junto con el nombre del modelo utilizado.

Referencing: cada fragmento contiene `doc_id`, que referencia el documento de conocimiento del cual se obtuvo. Los fragmentos se mantienen independientes para poder recuperarlos y compararlos por estrategia.

Ejemplo de documento:

```
{
  "_id": "ObjectId(...)",
  "doc_id": "ObjectId(documento_1)",
  "chunk_index": 3,
  "estrategia_chunking": "sentence-aware",
  "chunk_texto": "Fragmento de texto sobre una afección,
    sus síntomas o tratamientos.",
  "embedding": [0.023, -0.117, 0.204],
  "modelo": "jina-clip-v2",
  "fecha_ingesta": "ISODate(...)"
}
```

El vector es ilustrativo. Dado que se utiliza `jina-clip-v2`, cada *embedding* deberá tener 1024 dimensiones.

Justificación del modelo de *embeddings*: se seleccionó `jina-clip-v2` en lugar de `all-MiniLM-L6-v2` y de un modelo de texto independiente por las siguientes razones:

- **Espacio vectorial compartido:** el modelo representa texto e imágenes en un mismo espacio vectorial. Los fragmentos de `chunks` y las imágenes de `imagenes` pueden compararse directamente entre sí, lo que permite las consultas de texto a texto, texto a imagen, imagen a imagen e imagen a texto requeridas por el sistema.
- **Idioma:** los documentos de conocimiento están redactados en español, y all-MiniLM-L6-v2 está orientado principalmente al inglés. `jina-clip-v2` es un modelo multilingüe que incluye el español.
- **Longitud de los fragmentos:** las estrategias de *chunking* contemplan fragmentos de hasta 256 tokens. `jina-clip-v2` admite textos de hasta 8192 tokens, por lo que no trunca los fragmentos antes de generar el vector.
- **Costo y ejecución local:** sus pesos son de acceso abierto y pueden ejecutarse en un servidor propio, sin enviar el contenido de los documentos a un servicio externo. Su licencia (CC BY-NC 4.0) permite su uso en proyectos no comerciales, como este.

Consultas habilitadas: dado que `chunks` e `imagenes` comparten el mismo espacio vectorial, las consultas se resuelven de la siguiente manera:

- **Texto a texto:** el texto de la consulta se codifica y se busca en el índice vectorial de `chunks`.
- **Texto a imagen:** el texto de la consulta se codifica y se busca en el índice vectorial de `imagenes`.
- **Imagen a imagen:** la imagen de la consulta se codifica y se busca en el índice vectorial de `imagenes`.
- **Imagen a texto:** la imagen de la consulta se codifica y se busca en el índice vectorial de `chunks`.

Las estrategias que se compararán son:

- `fixed-size`**:** fragmentos de 256 tokens con solapamiento de 32 tokens.
- `sentence-aware`**:** fragmentos de máximo cinco oraciones, con solapamiento de una oración.
- `semantic`**:** agrupación de oraciones según su similitud semántica, con un umbral inicial de similitud coseno entre 0,75 y 0,85.

El campo `estrategia_chunking` es obligatorio porque permite identificar qué método produjo cada fragmento y comparar los resultados de recuperación obtenidos con las distintas estrategias.

5.2. Estrategias de indexación planificadas

En MongoDB, la indexación no se limita a las claves primarias (PK). También se crean índices sobre otros campos para acelerar las consultas que el sistema necesita realizar.

- **Clave primaria** `_id`**:** MongoDB crea automáticamente un índice único para este campo en cada colección.

- **Referencias:** campos como `diagnostico_id` o `paciente_id` pueden indexarse para acelerar búsquedas y relaciones entre colecciones.
- **Otros campos de consulta:** por ejemplo, `fecha_diagnostico` para consultar diagnósticos por fecha.
- **Búsqueda especializada:** campos como `embedding` necesitan índices vectoriales para realizar búsquedas por similitud; no son índices de claves primarias.

Se definirán índices según los patrones de consulta previstos. Se utilizarán índices simples y compuestos para consultas por identificadores, fechas y atributos frecuentes; índices de texto para búsqueda léxica; e índices vectoriales para recuperación semántica y multimodal. La Tabla 3 resume los índices planificados, agrupados por colección.

Cuadro 3: Índices planificados por colección

Índices planificados	**Finalidad**
`diagnosticos_dermatologicos`	
`nombre`: texto; `categoria`: ascendente	Buscar afecciones por nombre, descripción y categoría.
`sintomas`	
`nombre`: texto; `nombre`: ascendente y único	Evitar nombres duplicados y localizar síntomas.
`medicamentos`	
`nombre`: texto; `principio_activo`: ascendente	Buscar medicamentos por nombre o principio activo.
`tratamientos`	
`diagnosticos_ids`: ascendente; `medicamentos.medicamento_id`: ascendente	Recuperar tratamientos por diagnóstico o medicamento.
`diagnosticos_asignados`	
`paciente_id`: ascendente, `vigente`: ascendente y `fecha_diagnostico`: descendente; índice adicional sobre `diagnostico_id`	Consultar diagnósticos vigentes e históricos por paciente y afección.
`pacientes`	
`codigo_paciente`: ascendente y único	Identificar pacientes mediante un código único.
`historias_medicas`	
`paciente_id`: ascendente; `episodios.fecha`: descendente	Localizar historias clínicas y consultar episodios cronológicamente.
`imagenes`	
Índice vectorial sobre `embedding_visual` (1024 dimensiones); índices simples sobre `diagnostico_id` y `paciente_id`	Buscar imágenes similares y filtrar por diagnóstico o paciente.
`resultados_laboratorio`	
`paciente_id`: ascendente y `fecha`: descendente; índice adicional sobre `historia_medica_id`	Consultar exámenes por paciente, fecha e historia clínica.
`documentos_conocimiento`	
Índice de texto sobre `nombre_archivo` y `contenido_texto`	Recuperar documentos mediante coincidencias léxicas.

Continúa en la siguiente página

Cuadro 3: Índices planificados por colección (continuación)

Índices planificados	Finalidad
`chunks`	
Índice compuesto sobre `doc_id`, `estrategia_chunking` y `chunk_index`; índice vectorial sobre `embedding` (1024 dimensiones), con filtros sobre `estrategia_chunking` y `doc_id`	Localizar fragmentos, comparar estrategias de *chunking* y realizar recuperación semántica.

5.2.1. Consideraciones de implementación

- Los índices únicos se aplicarán únicamente a campos que deban identificar entidades sin duplicados. En `sintomas`, la unicidad de `nombre` supone normalizar previamente mayúsculas, tildes y variantes equivalentes.
- Los índices de texto tradicionales permiten coincidencias léxicas. En MongoDB Atlas también puede utilizarse Atlas Search para búsquedas textuales más configurables.
- Los índices vectoriales de `imagenes` y `chunks` se configurarán con similitud coseno y 1024 dimensiones, de acuerdo con el modelo `jina-clip-v2`. Los campos de filtro deberán declararse en la configuración del índice vectorial.
- Para las consultas sobre arreglos, MongoDB genera índices *multikey* cuando corresponde. Los índices compuestos se diseñarán evitando combinar múltiples campos de arreglo en un mismo índice.
- No se crearán índices para todos los campos indiscriminadamente: se revisarán los patrones de consulta y el costo de escritura y almacenamiento. El campo `_id` ya cuenta con índice automático.

5.3. Reglas de validación del esquema

Se utilizará el validador `\$jsonSchema` de MongoDB para comprobar los tipos, los campos obligatorios y las restricciones de los documentos. La validación se configurará con `validationAction: "error"` para rechazar documentos que incumplan las reglas establecidas.

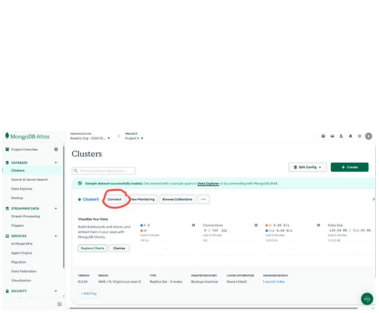
1. **Identificadores y referencias.** Los campos `_id` y las referencias como `paciente_id`, `diagnostico_id`, `tratamiento_id` y `doc_id` deberán utilizar el tipo BSON `objectId` cuando correspondan a identificadores MongoDB. Las referencias deberán apuntar a documentos existentes; esta comprobación se realizará desde la aplicación, ya que `\$jsonSchema` no valida relaciones entre colecciones.
2. **Campos obligatorios.** Cada colección exigirá los campos mínimos necesarios para identificar y describir su entidad. Por ejemplo: `nombre` en diagnósticos, síntomas y medicamentos; `codigo_paciente` en pacientes; `paciente_id` y `episodios` en historias médicas; `archivo` y `descripcion_visual` en imágenes; y `doc_id`, `chunk_texto`, `chunk_index` y `estrategia_chunking` en fragmentos.
3. **Tipos de datos.** Los nombres y las descripciones serán cadenas; las fechas serán BSON `date`; los indicadores de vigencia serán booleanos; y los identificadores múltiples se almacenarán en arreglos. Los campos numéricos de resultados de laboratorio deberán utilizar valores numéricos cuando el tipo de examen lo permita.

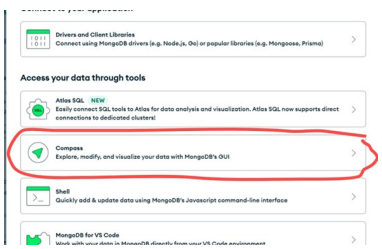
4. **Valores controlados.** El campo `estrategia_chunking` admitirá únicamente `fixed-size`, `sentence-aware` o `semantic`. El campo `formato` de los resultados de laboratorio se restringirá a los formatos efectivamente soportados, inicialmente `JSON` y `TXT`.
5. **Fechas y vigencia.** Los campos de fecha deberán contener fechas válidas o `null` cuando esto esté permitido. En `diagnosticos_asignados`, `fecha_fin` podrá ser nula mientras el tratamiento siga en curso. La aplicación comprobará que una fecha de fin no sea anterior a la fecha de inicio y que los estados de vigencia sean coherentes.
6. **Arreglos y documentos anidados.** `alergias`, `medicamentos`, `tratamientos_asignados`, `episodios`, `colores` y `texturas` deberán tener los tipos de elemento definidos para cada campo. Los objetos anidados deberán validar sus atributos obligatorios y tipos internos.
7. **Imágenes y vectores.** `embedding_visual` y `embedding` deberán contener arreglos numéricos de exactamente 1024 dimensiones. La aplicación comprobará también que todos sus valores sean finitos y que el vector corresponda al modelo registrado. El campo `paciente_id` podrá ser nulo para imágenes de referencia; cuando identifique un paciente, la aplicación comprobará la coherencia de sus relaciones clínicas.
8. **Fragmentos de conocimiento.** `chunk_index` deberá ser un entero no negativo y `chunk_texto` no podrá estar vacío. La combinación de `doc_id`, `estrategia_chunking` y `chunk_index` deberá ser única para evitar duplicar posiciones dentro de una misma estrategia de fragmentación.
9. **Integridad clínica.** La aplicación comprobará que las referencias a pacientes, diagnósticos, medicamentos, tratamientos, historias médicas e imágenes correspondan a registros válidos. También validará las relaciones entre alergias, tratamientos vigentes e interacciones documentadas antes de presentar posibles incompatibilidades.
10. **Calidad.** Se evitarán cadenas vacías en los campos descriptivos obligatorios y se normalizarán los valores utilizados para búsquedas.

6. Configuración de Entorno

6.1. Cluster MongoDB configurado *Atlas*

1. Instalamos el MongoCompass
2. Abrimos una cuenta en MongoAtlas
3. En Atlas guardamos la contraseña
4. En BaseData- clusters, apretamos “Conect”





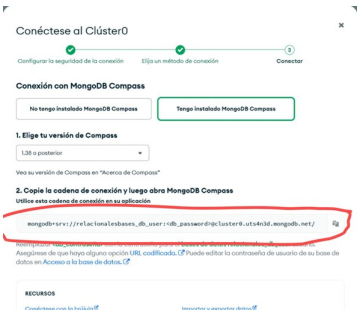


Figura 1: Imagenes Guía.

5. El link lo pegamos en MongoCompass

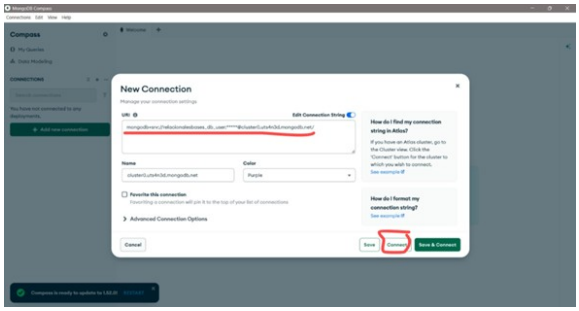


Figura 2: Imagen Guía.

6. Con esto verificamos que esta conectada

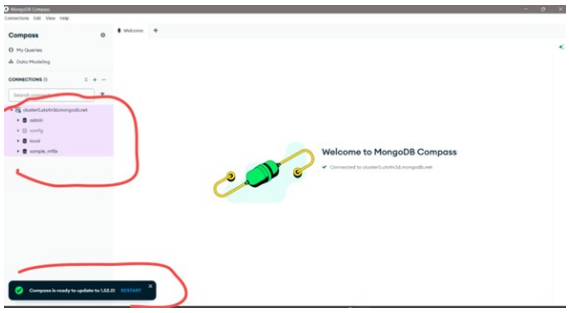


Figura 3: Imagen Guía.

6.2. Scripts de inicialización

Para nuestro proyecto utilizaremos como ejecutable de codigo *COLAB*, Donde inicilizaremos la base de datos creando la conexión en con MongoDB. Crearemos las 11 colecciones propuestas así como sus indices correspondientes.

1. ponemos el siguiente comando: *!!pip install pymongo dnspython* y tiene que cargarnos así

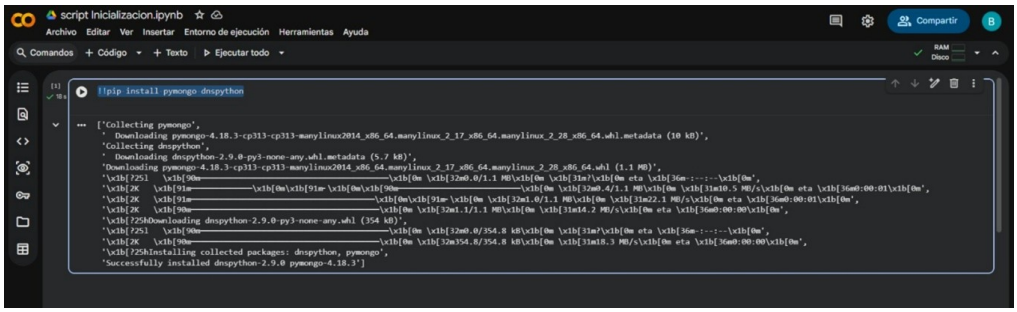


Figura 4: Imagen Guía.

2. Luego ingresamos este bloque de código que contiene funciones de inicialización y verificación



Figura 5: Imagenes Guía.

3. Nos tiene que salir este mensaje de éxito

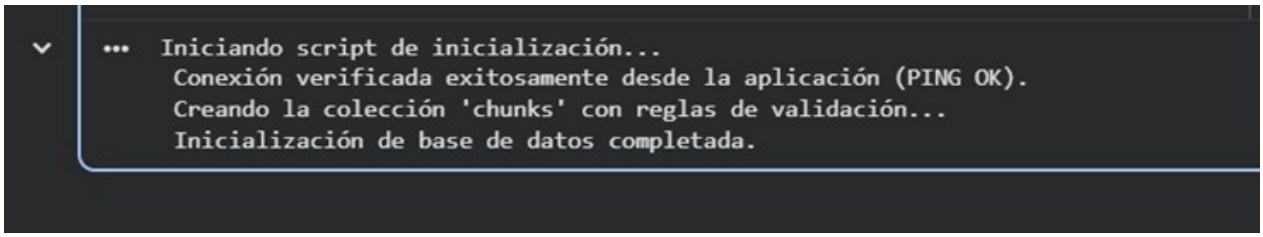


Figura 6: Imagen Guía.

6.3. Conexión verificada desde aplicación

1. En compass refrescamos para asegurarnos que funciono la conexión

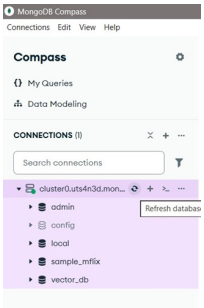


Figura 7: Imagen Guía.

2. Nos aparece estos, entonces confirmamos que funciono

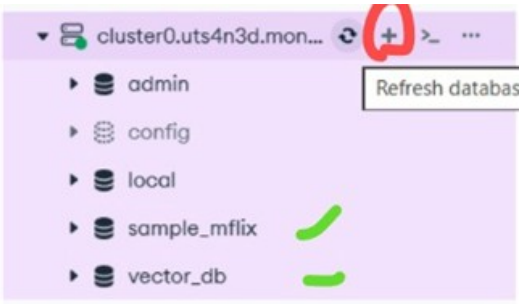


Figura 8: Imagen Guía.

6.4. Script de Colecciones Ejemplo insertadas en Mongo

```
def poblar_base_de_datos():
    client = obtener_cliente()
    db = client[DB_NAME]

    print(" Creando e insertando datos de ejemplo en las 11 colecciones
        ...\n")

    # Generación de IDs compartidos para mantener la integridad
        referencial
    sintoma_1_id = ObjectId()
    sintoma_2_id = ObjectId()
    medicamento_1_id = ObjectId()
    tratamiento_1_id = ObjectId()
    diagnostico_1_id = ObjectId()
    paciente_1_id = ObjectId()
    historia_1_id = ObjectId()
    diag_asignado_1_id = ObjectId()
    doc_conocimiento_1_id = ObjectId()
    imagen_1_id = ObjectId()
    resultado_1_id = ObjectId()

    # 1. Colección: sintomas
    sintomas = [
        {
            "_id": sintoma_1_id,
            "nombre": "Picazón",
            "descripcion": "Sensación de irritación que provoca la
                necesidad de rascarse."
        },
        {
            "_id": sintoma_2_id,
            "nombre": "Descamación",
            "descripcion": "Desprendimiento de laminillas córneas de la
                epidermis."
        }
    ]
    db["sintomas"].delete_many({})
    db["sintomas"].insert_many(sintomas)
    print(" 1/11 Colección 'sintomas' lista.")

    # 2. Colección: diagnosticos_dermatologicos
    diagnosticos = [
        {
            "_id": diagnostico_1_id,
            "nombre": "Psoriasis",
            "categoria": "Inflamatoria y autoinmune",
            "descripcion": "Afección inflamatoria crónica de la piel.",
            "sintomas_ids": [sintoma_1_id, sintoma_2_id],
            "manifestaciones_visuales": ["Placas rojizas", "Escamas
                blanquecinas"],
```

```
            "tratamientos_ids": [tratamiento_1_id]
        }
    ]
    db["diagnosticos_dermatologicos"].delete_many({})
    db["diagnosticos_dermatologicos"].insert_many(diagnosticos)
    print(" 2/11 Colección 'diagnosticos_dermatologicos' lista.")

    # 3. Colección: medicamentos
    medicamentos = [
        {
            "_id": medicamento_1_id,
            "nombre": "Clobetasol Propionato",
            "principio_activo": "Clobetasol",
            "indicaciones": ["Tratamiento a corto plazo de dermatosis
                inflamatorias"],
            "contraindicaciones": ["Infecciones cutáneas no tratadas", "
                Rosácea"],
            "interacciones": [
                {
                    "medicamento_relacionado_id": ObjectId(),
                    "descripcion": "Aumento del riesgo de supresión
                        suprarrenal si se combina con otros corticoides."
                }
            ]
        }
    ]
    db["medicamentos"].delete_many({})
    db["medicamentos"].insert_many(medicamentos)
    print(" 3/11 Colección 'medicamentos' lista.")

    # 4. Colección: tratamientos
    tratamientos = [
        {
            "_id": tratamiento_1_id,
            "nombre": "Tratamiento tópico de Psoriasis",
            "descripcion": "Tratamiento de aplicación local con
                corticoides de alta potencia.",
            "diagnosticos_ids": [diagnostico_1_id],
            "indicaciones": ["Aplicar una delgada capa en la zona afectada
                "],
            "precauciones": ["No usar por más de 2 semanas continuas"],
            "medicamentos": [
                {
                    "medicamento_id": medicamento_1_id,
                    "pautas_generales": [
                        {
                            "grupo_poblacional": "Adulto",
                            "cantidad": "1 o 2 aplicaciones diarias",
                            "frecuencia": "Cada 12 a 24 horas"
                        },
                        {
```

```
                        "grupo_poblacional": "Pediátrico",
                        "cantidad": "Uso no recomendado o bajo estricta
                            supervisión",
                        "frecuencia": "Según criterio médico"
                    }
                ]
            }
        ]
    }
]
db["tratamientos"].delete_many({})
db["tratamientos"].insert_many(tratamientos)
print(" 4/11 Colección 'tratamientos' lista.")

# 5. Colección: diagnosticos_asignados
diagnosticos_asignados = [
    {
        "_id": diag_asignado_1_id,
        "paciente_id": paciente_1_id,
        "historia_medica_id": historia_1_id,
        "episodio_fecha": datetime.datetime(2026, 3, 15, 10, 30),
        "diagnostico_id": diagnostico_1_id,
        "fecha_diagnostico": datetime.datetime(2026, 3, 15),
        "vigente": True,
        "observaciones": "Brote moderado en codos y rodillas.",
        "tratamientos_asignados": [
            {
                "tratamiento_id": tratamiento_1_id,
                "fecha_inicio": datetime.datetime(2026, 3, 15),
                "fecha_fin": None,
                "vigente": True,
                "medicamentos": [
                    {
                        "medicamento_id": medicamento_1_id,
                        "cantidad": "1 aplicación fina en codos",
                        "frecuencia": "Cada 12 horas por 10 días"
                    }
                ],
                "observaciones": "Reevaluar en 10 días."
            }
        ]
    }
]
db["diagnosticos_asignados"].delete_many({})
db["diagnosticos_asignados"].insert_many(diagnosticos_asignados)
print(" 5/11 Colección 'diagnosticos_asignados' lista.")

# 6. Colección: pacientes
pacientes = [
    {
        "_id": paciente_1_id,
```

```
            "codigo_paciente": "PAC-001",
            "nombre_completo": "Juan Pérez",
            "fecha_nacimiento": datetime.datetime(1990, 5, 20),
            "lugar_residencia": {
                "pais": "Colombia",
                "ciudad": "Manizales"
            },
            "fecha_defuncion": None,
            "alergias": [
                {
                    "sustancia": "Penicilina",
                    "reaccion": "Urticaria y rash cutáneo"
                }
            ],
            "diagnosticos_asignados_ids": [diag_asignado_1_id],
            "historias_clinicas_ids": [historia_1_id]
        }
    ]
    db["pacientes"].delete_many({})
    db["pacientes"].insert_many(pacientes)
    print(" 6/11 Colección 'pacientes' lista.")

    # 7. Colección: historias_medicas
    historias_medicas = [
        {
            "_id": historia_1_id,
            "paciente_id": paciente_1_id,
            "episodios": [
                {
                    "fecha": datetime.datetime(2026, 3, 15, 10, 30),
                    "diagnosticos_asignados_ids": [diag_asignado_1_id],
                    "observaciones": "Paciente presenta placas
                        eritematodescamativas.",
                    "imagenes_ids": [imagen_1_id],
                    "resultados_laboratorio_ids": [resultado_1_id]
                }
            ]
        }
    ]
    db["historias_medicas"].delete_many({})
    db["historias_medicas"].insert_many(historias_medicas)
    print(" 7/11 Colección 'historias_medicas' lista.")

    # 8. Colección: imagenes
    imagenes = [
        {
            "_id": imagen_1_id,
            "paciente_id": paciente_1_id,
            "historia_medica_id": historia_1_id,
            "episodio_fecha": datetime.datetime(2026, 3, 15, 10, 30),
            "diagnostico_id": diagnostico_1_id,
```

```
            "archivo": "lesion_codo_001.jpg",
            "descripcion_visual": {
                "colores": ["Rojizo", "Blanquecino"],
                "texturas": ["Descamación"],
                "region_corporal": "Codo"
            },
            "embedding_visual": [0.12, -0.08, 0.31, 0.05, -0.19], # Vector
                ilustrativo
            "fecha_registro": datetime.datetime(2026, 3, 15, 11, 0)
        }
    ]
    db["imagenes"].delete_many({})
    db["imagenes"].insert_many(imagenes)
    print(" 8/11 Colección 'imagenes' lista.")

    # 9. Colección: resultados_laboratorio
    resultados_laboratorio = [
        {
            "_id": resultado_1_id,
            "paciente_id": paciente_1_id,
            "historia_medica_id": historia_1_id,
            "fecha": datetime.datetime(2026, 3, 15, 11, 15),
            "tipo_examen": "Biopsia de piel",
            "formato": "JSON",
            "resultado": {
                "parametro": "Hiperqueratosis con paraqueratosis",
                "valor": "Positivo",
                "observaciones": "Hallazgos compatibles con psoriasis
                    vulgar."
            }
        }
    ]
    db["resultados_laboratorio"].delete_many({})
    db["resultados_laboratorio"].insert_many(resultados_laboratorio)
    print(" 9/11 Colección 'resultados_laboratorio' lista.")

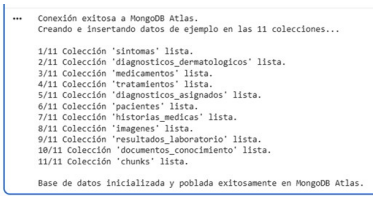
    # 10. Colección: documentos_conocimiento
    documentos_conocimiento = [
        {
            "_id": doc_conocimiento_1_id,
            "nombre_archivo": "guia_psoriasis_2026.txt",
            "tipo_documento": "Texto",
            "idioma": "es",
            "contenido_texto": "La psoriasis es una enfermedad
                inflamatoria crónica de la piel caracterizada por placas
                eritematosas con escamas plateadas.",
            "fecha": datetime.datetime(2026, 1, 10)
        }
    ]
    db["documentos_conocimiento"].delete_many({})
    db["documentos_conocimiento"].insert_many(documentos_conocimiento)
```

```
        print(" 10/11 Colección 'documentos_conocimiento' lista.")

        # 11. Colección: chunks (con embedding vectorial de jina-clip-v2)
        # Nota: jina-clip-v2 usa 1024 dimensiones. Simulamos 5 valores
            flotantes para el ejemplo.
        chunks = [
            {
                "_id": ObjectId(),
                "doc_id": doc_conocimiento_1_id,
                "chunk_index": 0,
                "estrategia_chunking": "sentence-aware",
                "chunk_texto": "La psoriasis es una enfermedad inflamatoria cr
                    ónica de la piel caracterizada por placas eritematosas.",
                "embedding": [0.023, -0.117, 0.204, 0.089, -0.045],
                "modelo": "jina-clip-v2",
                "fecha_ingesta": datetime.datetime(2026, 3, 16)
            }
        ]
        db["chunks"].delete_many({})
        db["chunks"].insert_many(chunks)
        print(" 11/11 Colección 'chunks' lista.")

        print("\n Base de datos inicializada y poblada exitosamente en
            MongoDB Atlas.")

    if __name__ == "__main__":
        poblar_base_de_datos()
```





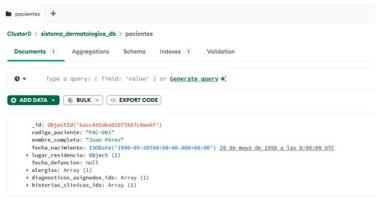


Figura 9: Imagenes de Confirmación.

6.5. Implementar la indexación de las colecciones.

Se establece que los embeddings definitivos tendrán una dimensionalidad de 1024 dimensiones, de acuerdo con el modelo *jina-clip-v2*. Sin embargo, durante esta etapa inicial de desarrollo se utilizarán vectores de prueba de 5 dimensiones para validar la estructura de las colecciones y el proceso de almacenamiento de datos. Una vez confirmada esta estruc-

tura e integrado el modelo, estos vectores serán reemplazados por embeddings reales de 1024 dimensiones para configurar los índices vectoriales definitivos.

```
from pymongo import ASCENDING, DESCENDING, TEXT, MongoClient

def crear_indices():
    """Crea los índices convencionales planificados para el proyecto."""

    with MongoClient(MONGO_URI) as client:
        db = client[DB_NAME]

        indices = [
            # 1. Diagnósticos dermatológicos
            (
                "diagnosticos_dermatologicos",
                [("nombre", TEXT), ("descripcion", TEXT)],
                {"name": "idx_diagnosticos_texto"}
            ),
            (
                "diagnosticos_dermatologicos",
                [("categoria", ASCENDING)],
                {"name": "idx_diagnosticos_categoria"}
            ),

            # 2. Síntomas
            (
                "sintomas",
                [("nombre", TEXT)],
                {"name": "idx_sintomas_texto"}
            ),
            (
                "sintomas",
                [("nombre", ASCENDING)],
                {"name": "idx_sintomas_nombre_unico", "unique": True}
            ),

            # 3. Medicamentos
            (
                "medicamentos",
                [("nombre", TEXT)],
                {"name": "idx_medicamentos_texto"}
            ),
            (
                "medicamentos",
                [("principio_activo", ASCENDING)],
                {"name": "idx_medicamentos_principio_activo"}
            ),

            # 4. Tratamientos
            (
                "tratamientos",
                [("diagnosticos_ids", ASCENDING)],
```

```
            {"name": "idx_tratamientos_diagnosticos"}
        ),
        (
            "tratamientos",
            [("medicamentos.medicamento_id", ASCENDING)],
            {"name": "idx_tratamientos_medicamentos"}
        ),

        # 5. Diagnósticos asignados a pacientes
        (
            "diagnosticos_asignados",
            [
                ("paciente_id", ASCENDING),
                ("vigente", ASCENDING),
                ("fecha_diagnostico", DESCENDING)
            ],
            {"name": "idx_diag_asignados_paciente_vigencia_fecha"}
        ),
        (
            "diagnosticos_asignados",
            [("diagnostico_id", ASCENDING)],
            {"name": "idx_diag_asignados_diagnostico"}
        ),

        # 6. Pacientes
        (
            "pacientes",
            [("codigo_paciente", ASCENDING)],
            {"name": "idx_pacientes_codigo_unico", "unique": True}
        ),

        # 7. Historias médicas
        (
            "historias_medicas",
            [
                ("paciente_id", ASCENDING),
                ("episodios.fecha", DESCENDING)
            ],
            {"name": "idx_historias_paciente_fecha"}
        ),

        # 8. Imágenes
        (
            "imagenes",
            [("diagnostico_id", ASCENDING)],
            {"name": "idx_imagenes_diagnostico"}
        ),
        (
            "imagenes",
            [("paciente_id", ASCENDING)],
            {"name": "idx_imagenes_paciente"}
```

```
                ),

                # 9. Resultados de laboratorio
                (
                    "resultados_laboratorio",
                    [
                        ("paciente_id", ASCENDING),
                        ("fecha", DESCENDING)
                    ],
                    {"name": "idx_laboratorio_paciente_fecha"}
                ),
                (
                    "resultados_laboratorio",
                    [("historia_medica_id", ASCENDING)],
                    {"name": "idx_laboratorio_historia"}
                ),

                # 10. Documentos de conocimiento
                (
                    "documentos_conocimiento",
                    [("nombre_archivo", TEXT), ("contenido_texto", TEXT)],
                    {"name": "idx_documentos_texto"}
                ),

                # 11. Fragmentos para RAG
                (
                    "chunks",
                    [
                        ("doc_id", ASCENDING),
                        ("estrategia_chunking", ASCENDING),
                        ("chunk_index", ASCENDING)
                    ],
                    {"name": "idx_chunks_documento_estrategia_orden"}
                )
            ]

            for coleccion, campos, opciones in indices:
                nombre = db[coleccion].create_index(campos, **opciones)
                print(f"ndice creado/verificado: {coleccion}.{nombre}")

            print("\nIndexación convencional completada.")


    crear_indices()
```

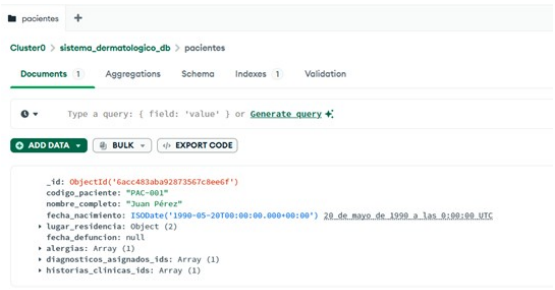


Figura 10: Imagen de Confirmación.

7. Plan de experimento de *chunking*

Los tres métodos (fixed-size, sentence-aware , semantic) se aplican a la misma colección `documentos_conocimiento` y sus fragmentos se guardan en `chunks`, identificados por el campo obligatorio `estrategia_chunking`.

7.1. Definición de las estrategias a comparar y sus parámetros

Cuadro 4: Estrategias de *chunking* a comparar

Estrategia	**Funcionamiento y parámetros**	**Ventaja / riesgo esperado**
`fixed-size`	Ventanas de 256 tokens con solapamiento de 32 tokens. Los tokens se cuentan con el tokenizador de `jina-clip-v2`.	Simple y predecible. Puede cortar frases, listas o tablas por la mitad.
`sentence-aware`	Fragmentos de máximo 5 oraciones con solapamiento de 1 oración. Las oraciones se segmentan con un segmentador para español (spaCy `es_core_news_sm` o NLTK `punkt`).	Conserva frases completas. El tamaño en tokens es variable y puede mezclar temas.
`semantic`	Se genera el *embedding* de cada oración con `jina-clip-v2` y se abre un nuevo fragmento cuando la similitud coseno entre oraciones consecutivas cae por debajo del umbral (inicialmente entre 0,75 y 0,85). Tope de 256 tokens por fragmento para que la comparación sea justa.	Fragmentos coherentes por tema. Mayor costo de cómputo y sensibilidad al umbral.

Para que la única variable sea la estrategia de *chunking*, se mantienen constantes la base de conocimiento, el modelo de *embeddings*, la recuperación, el LLM con su *prompt* y las consultas. (El prompt esta pendiente por definir):

- **Base de conocimiento:** los mismos documentos de `documentos_conocimiento` para las tres estrategias.
- **Modelo de *embeddings*:** `jina-clip-v2` (1024 dimensiones, similitud coseno).
- **Recuperación:** la misma búsqueda vectorial, limitada a los fragmentos de cada estrategia y con el mismo número de fragmentos recuperados.
- **Generación:** el mismo modelo de lenguaje con el mismo *prompt*.
- **Consultas:** las mismas 10 consultas para las tres estrategias.

Las métricas con las que se evaluará cada estrategia se definen en la Tabla 5.

Cuadro 5: Métricas de evaluación de las estrategias

Métrica	Definición
Número de fragmentos	Total de fragmentos generados por estrategia sobre todos los documentos.
Longitud promedio del fragmento	Promedio de tokens por fragmento, por estrategia.
Calidad percibida	Puntaje de 1 a 5 que cada integrante asigna a la respuesta generada, según sea relevante, completa y fundamentada en los fragmentos recuperados. Se reporta el promedio.

7.2. Conjunto de 10 consultas de prueba seleccionadas

Las 10 consultas cubren las cinco categorías del universo del discurso (inflamatorias, infecciosas, anexos cutáneos, neoplásicas y genéticas) y distintos tipos de pregunta, de modo que cada estrategia se pruebe en situaciones favorables y desfavorables. Se presentan en la Tabla 6.

Cuadro 6: Consultas de prueba para el experimento de *chunking*

ID	Consulta	Qué pone a prueba	Requerimiento
C1	¿Qué enfermedades de la piel producen placas rojizas con escamas blanquecinas en codos y rodillas?	Síntomas a afección; dato localizado en un párrafo.	RF-01, RF-07
C2	¿Cuál es el tratamiento tópico recomendado para la dermatitis atópica en niños?	Tratamiento y pauta pediátrica.	RF-02
C3	¿Por qué la epidermólisis ampollosa produce ampollas con traumatismos leves y cómo se cuidan las heridas?	Causa genética y cuidados en el mismo documento.	RF-02, RF-07

Continúa en la siguiente página

Cuadro 6: Consultas de prueba para el experimento de *chunking* (continuación)

ID	Consulta	Qué pone a prueba	Requerimiento
C4	¿En qué se diferencian el carcinoma basocelular y el carcinoma espinocelular?	Comparación entre dos temas; riesgo de mezclar temas en un mismo fragmento.	RF-07, RF-08
C5	¿Qué características de una lesión pigmentada sugieren melanoma según la regla ABCDE?	Lista enumerativa que puede quedar partida entre fragmentos.	RF-07, RF-08
C6	¿Qué infecciones cutáneas se transmiten por contacto directo y cómo se tratan?	Información repartida en varios documentos (impétigo, escabiosis, tiña, verrugas).	RF-02, RF-07
C7	Una paciente alérgica a la penicilina tiene impétigo, ¿qué antibióticos debería evitar?	Contraindicaciones y alergias.	RF-05, RF-08
C8	¿Cómo se distingue visualmente la rosácea del acné?	Manifestaciones visuales de dos afecciones.	RF-07, RF-08
C9	¿Qué factores desencadenan los brotes de lupus eritematoso cutáneo y cómo se maneja la enfermedad?	Dos subtemas (causas y manejo) dentro de un mismo documento.	RF-07, RF-08
C10	Me salen manchas blancas en la cara y las manos que no duelen, ¿qué podría ser?	Lenguaje coloquial; robustez semántica sin términos técnicos.	RF-01, RF-07